

React.js进阶

渲染数组

我们有一个列表

```
<ul>
  <li>凯瑟琳·约翰逊：数学家</li>
  <li>马里奥·莫利纳：化学家</li>
  <li>穆罕默德·阿卜杜勒·萨拉姆：物理学家</li>
  <li>珀西·莱温·朱利亚：化学家</li>
  <li>苏布拉马尼扬·钱德拉塞卡：天体物理学家</li>
</ul>
```

可以看到，这些列表项之间唯一的区别就是其中的内容/数据。未来你可能会碰到很多类似的情况，在那些场景中，你想基于不同的数据渲染出相似的组件，比如评论列表或者个人资料的图库。在这样的场景下，可以把要用到的数据存入 JavaScript 对象或数组，然后用 `map()` 或 `filter()` 这样的方法来渲染出一个组件列表。

把数据存储到数组中

```
export default function List() {  
  const people = [  
    {  
      id: 0,  
      name: '凯瑟琳·约翰逊',  
      profession: '数学家',  
      accomplishment: '太空飞行相关数值的核算',  
      imageId: 'MK3eW3A',  
    },  
    {  
      id: 1,  
      name: '马里奥·莫利纳',  
      profession: '化学家',  
      accomplishment: '北极臭氧空洞的发现',  
      imageId: 'mynHUSa',  
    },  
    {  
      id: 2,  
      name: '穆罕默德·阿卜杜勒·萨拉姆',  
      profession: '物理学家',  
      accomplishment: '关于基本粒子间弱相互作用和电磁相互作用的统一理论',  
      imageId: 'bE7W1ji',  
    },  
    {  
      id: 3,  
      name: '珀西·莱温·朱利亚',  
      profession: '化学家',  
      accomplishment: '开创性的可的松药物、类固醇和避孕药的研究',  
      imageId: 'IOjWm71',  
    },  
    {  
      id: 4,  
      name: '苏布拉马尼扬·钱德拉塞卡',  
      profession: '天体物理学家',  
      accomplishment: '白矮星质量计算',  
      imageId: 'lrWQx81',  
    },  
  ],  
  
  const chemists = people.filter(person =>  
    person.profession === '化学家'
```

```

);

const listItems = chemists.map(person =>
  <li>
    <img
      src={'https://img.picui.cn/free/2024/10/16/670fd0c5a6ab6.png'}
      alt={person.name}
      width={100}
    />
    <p>
      <b>{person.name}</b>
      { ' ' + person.profession + ' ' }
      因{person.accomplishment}而闻名世界
    </p>
  </li>
);

return <ul>{listItems}</ul>;
}

```

用 key 保持列表项的顺序

你会发现控制台有这样一个报错

```

Warning: Each child in a list should have a unique "key" prop.

Check the top-level render call using <ul>. See https://reactjs.org/link/warning-keys for more information.
    at li

```

这是因为你必须给数组中的每一项都指定一个 `key` ——它可以是字符串或数字的形式，只要能唯一标识出各个数组项就行

直接放在 `map()` 方法里的 JSX 元素一般都需要指定 `key` 值！

```
<li key={person.id}>...</li>
```

这些 `key` 会告诉 React，每个组件对应着数组里的哪一项，所以 React 可以把它们匹配起来。这在数组项进行移动（例如排序）、插入或删除等操作时非常重要。一个合适的 `key` 可以帮助 React 推断发生了什么，从而得以正确地更新 DOM 树。

React 中为什么需要 key?

设想一下，假如你桌面上的文件都没有文件名，取而代之的是，你需要通过文件的位置顺序来区分它们——第一个文件，第二个文件，以此类推。也许你也不是不能接受这种方式，可是一旦你删除了其中

的一个文件，这种组织方式就会变得混乱无比。原来的第二个文件可能会变成第一个文件，第三个文件会成为第二个文件.....

React 里需要 key 和文件夹里的文件需要有文件名的道理是类似的。它们（key 和文件名）都让我们可以从众多的兄弟元素中唯一标识出某一项（JSX 节点或文件）。而一个精心选择的 key 值所能提供的信息远远不止于这个元素在数组中的位置。即使元素的位置在渲染的过程中发生了改变，它提供的 key 值也能让 React 在整个生命周期中一直认得它。

不要随意生成 key

你可能会想直接把数组项的索引当作 key 值来用，实际上，如果你没有显式地指定 key 值，React 确实默认会这么做。但是数组项的顺序在插入、删除或者重新排序等操作中会发生改变，此时把索引顺序用作 key 值会产生一些微妙且令人困惑的 bug。

与之类似，请不要在运行过程中动态地产生 key，像是 `key={Math.random()}` 这种方式。这会导致每次重新渲染后的 key 值都不一样，从而使得所有的组件和 DOM 元素每次都要重新创建。这不仅会造成运行变慢的问题，更有可能导致用户输入的丢失。所以，使用能从给定数据中稳定取得的值才是明智的选择。

JSON

- **JavaScript Object Notation**(JavaScript 对象表示法)
- 是轻量级的文本数据交换格式
- JSON 独立于语言：JSON 使用 Javascript 语法来描述数据对象，但是 JSON 仍然独立于语言 and 平台。JSON 解析器和 JSON 库支持许多不同的编程语言。

前后端交互时，后端API经常会返回JSON格式的数据。

示例

```
{
  "sites": [
    { "name": "菜鸟教程" , "url": "www.runoob.com" },
    { "name": "google" , "url": "www.google.com" },
    { "name": "微博" , "url": "www.weibo.com" }
  ]
}
```

读取

如果你的JSON文件是本地的，并且作为项目的静态资源，你可以直接导入它，并用 `JSON.parse()` 函数来解释它，使其变为JS对象，相反的你也可以用 `JSON.stringify()` 来把JS对象变为JSON格式，从而与服务器（后端）进行交互

```
import data from './data.json'; // 直接导入JSON文件

function DataDisplay() {
  const obj = JSON.parse(data)
  return <div>{obj.sites[0].name}</div>;// JSON.stringify(data)得到一个JS对象
}
```

useRef

简介

当你希望组件“记住”某些信息，但又不想让这些信息触发新的渲染时，你可以使用 `ref`。

我们可以回忆 `useState` 解决了什么问题

1. 局部变量无法在多次渲染中持久保存。当 React 再次渲染这个组件时，它会从头开始渲染——不会考虑之前对局部变量的任何更改。
2. 更改局部变量不会触发渲染。React 没有意识到它需要使用新数据再次渲染组件。

`useRef` 则是解决1没解决2，也就是说，重新渲染时 `ref` 会保存，但是 `ref` 的改变不会导致页面重新渲染。

用法

导入 `useRef`

```
import { useRef } from 'react';
```

在你的组件内，调用 `useRef` 并传入你想要引用的初始值作为唯一参数。例如，这里的 `ref` 引用的值是“0”：

```
const ref = useRef(0);
```

useRef 返回一个这样的对象:

```
{  
  current: 0 // 你向 useRef 传入的值  
}
```

你可以用 `ref.current` 属性访问该 `ref` 的当前值。这个值是有意被设置为可变的（不同于 `state`）

显然使用 `useRef` 的开销是要比 `useState` 要小的，因为渲染的频率降低了

我们来看下面的例子

```

'use client';
import { useState, useRef } from 'react';

export default function Stopwatch() {
  const [startTime, setStartTime] = useState<number | null>(null);
  const [now, setNow] = useState<number | null>(null);
  const intervalRef = useRef<NodeJS.Timeout | number>();

  function handleStart() {
    setStartTime(Date.now());
    setNow(Date.now());
    clearInterval(intervalRef.current);
    intervalRef.current = setInterval(() => {
      setNow(Date.now());
    }, 10);
  }

  function handleStop() {
    clearInterval(intervalRef.current);
  }

  let secondsPassed = 0;
  if (startTime != null && now != null) {
    secondsPassed = (now - startTime) / 1000;
  }

  return (
    <>
      <h1>时间过去了: {secondsPassed.toFixed(3)}</h1>
      <button onClick={handleStart}>
        开始
      </button>
      <button onClick={handleStop}>
        停止
      </button>
    </>
  );
}

```

由于 interval ID 不用于渲染，你可以将其保存在 ref 中
[MDN setInterval函数文档](#)

使用 ref 操作 DOM

由于 React 会自动更新 DOM 以匹配渲染输出，因此组件通常不需要操作 DOM。但是，有时可能需要访问由 React 管理的 DOM 元素——例如聚焦节点、滚动到此节点，以及测量它的尺寸和位置。React 没有内置的方法来执行此类操作，所以需要有一个指向 DOM 节点的 ref 来实现。例如，点击按钮将使用 ref 聚焦输入框：

```
'use client';
import { useRef } from 'react';

export default function Form() {
  const inputRef = useRef<HTMLInputElement>(null);

  function handleClick() {
    inputRef.current?.focus();
  }

  return (
    <>
      <input ref={inputRef} />
      <button onClick={handleClick}>
        聚焦输入框
      </button>
    </>
  );
}
```

DOM 操作是 ref 最常见的用例。

ref 不能在渲染时操作 DOM

```
'use client';
import { useState, useRef, useEffect } from 'react';

function VideoPlayer({ src, isPlaying }: {src:string,isPlaying:boolean}) {
  const ref = useRef(null);

  if (isPlaying) {
    ref.current.play(); // 渲染期间不能调用 `play()`。
  } else {
    ref.current.pause(); // 同样，调用 `pause()` 也不行。
  }

  return <video ref={ref} src={src} loop playsInline />;
}

export default function App() {
  const [isPlaying, setIsPlaying] = useState(false);
  return (
    <>
      <button onClick={() => setIsPlaying(!isPlaying)}>
        {isPlaying ? '暂停' : '播放'}
      </button>
      <VideoPlayer
        isPlaying={isPlaying}
        src="https://interactive-examples.mdn.mozilla.net/media/cc0-videos/flower.mp4"
      />
    </>
  );
}
```

Unhandled Runtime Error

TypeError: Cannot read properties of null (reading 'pause')

Source

app\page.tsx (10:17) @ pause



```
8 |     ref.current.play(); // 渲染期间不能调用 `play()`。
9 |   } else {
> 10 |     ref.current.pause(); // 同样，调用 `pause()` 也不行。
    |                      ^
11 |   }
12 |
13 |   return <video ref={ref} src={src} loop playsInline />;
```

Call Stack

Show collapsed frames

可以使用 `useEffect` 包裹副作用，把它分离到渲染逻辑的计算过程之外。

useEffect

纯函数与副作用

`useEffect` 中的 `Effect` 对应英文中的 `side effect`，也就是副作用的意思

什么是纯函数：

- 确定性：输入相同时输出也相同。
- 无副作用：只负责自己的任务。它不会更改在该函数调用前就已存在的对象状态或变量。

```

let guest: number = 0;

function Cup() {
  // Bad: changing a preexisting variable!
  guest = guest + 1;
  return <h2>Tea cup for guest #{guest}</h2>;
}

export default function TeaSet() {
  return (
    <>
      <Cup />
      <Cup />
      <Cup />
    </>
  );
}

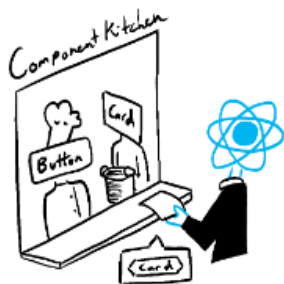
```

React组件的设计理念就是纯函数。（输入相同时输出也相同？）

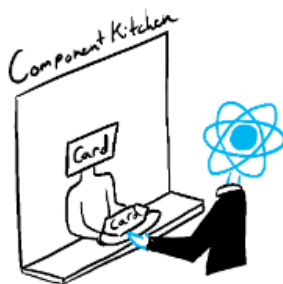
网络连接就是一种典型的副作用。它可能会改变服务器上的数据，或者在客户端和服务端之间建立一个新的连接状态。

React 提供页面的三个步骤

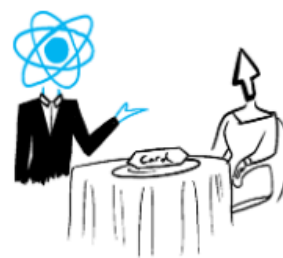
1. 触发一次渲染（把客人的点单分发到厨房）
2. 渲染组件（在厨房准备订单）
3. 提交到DOM（将菜品放在桌子上）



触发



渲染



提交

分的更细的话

- 组件渲染：React 会根据组件的props和state生成一个虚拟DOM树。
- 虚拟DOM diffing：React将生成的虚拟DOM树与前一个虚拟DOM树进行比较，这个过程称为“diffing”。React的算法会计算出最小数量的DOM操作来更新实际的DOM，使得页面的内容与虚拟DOM树保持一致。
- 实际DOM更新：根据diffing过程的结果，React会执行必要的DOM操作来更新浏览器中的实际DOM。这个过程也是同步的，**一旦完成，组件的内容就会显示在屏幕上。**
- useEffect执行：在组件内容更新到屏幕之后，React会执行useEffect中的代码。这意味着useEffect中的代码（如数据获取、订阅或手动DOM操作等）会在组件内容已经显示在屏幕上之后执行。

为什么要用 useEffect

在React中，理想的组件应该是独立和可复用的。

- 手动对DOM进行操作时。
- 副作用会使组件的渲染变慢，因为副作用不一定要放在渲染过程中
- 将副作用放在组件内部会使得组件依赖于特定的外部条件，从而降低其独立性和可复用性。
- memo （后面会讲）

Effect 允许你指定由渲染本身，而不是特定事件引起的副作用。Effect 在屏幕更新后的提交阶段运行。

声明 useEffect

首先在 React 中引入 useEffect Hook：

```
import { useEffect } from 'react';
```

然后以函数的形式传入你想执行的副作用

```
useEffect(() => {  
  if (isPlaying) {  
    ref.current!.play();  
  } else {  
    ref.current!.pause();  
  }  
})//注：!为非空断言操作符
```

第二个参数为空时，`useEffect` 将在每次渲染、屏幕更新之后调用
显然，`useEffect` 不会导致屏幕的更新，否则就陷入死循环

指定 Effect 依赖

一般来说，`useEffect` 会在每次渲染时执行。但更多时候，并不需要每次渲染的时候都执行 `useEffect`。

有时这会导致程序逻辑错误。例如，组件的淡入动画只需要在第一轮渲染出现时播放一次，而不是每次触发新一轮渲染后都播放。

将**依赖数组**传入 `useEffect` 的第二个参数，以告诉 React 跳过不必要地重新运行 Effect。

```

"use client";
import { useState, useRef, useEffect } from 'react';

function VideoPlayer({ src, isPlaying }: { src: string, isPlaying: boolean }) {
  const ref = useRef<HTMLVideoElement>(null);

  useEffect(() => {
    if (isPlaying) {
      console.log('Calling video.play()');
      ref.current!.play();
    } else {
      console.log('Calling video.pause()');
      ref.current!.pause();
    }
  }, [isPlaying]);

  return <video ref={ref} src={src} loop playsInline />;
}

export default function App() {
  const [isPlaying, setIsPlaying] = useState(false);
  const [text, setText] = useState('');
  return (
    <>
      <input value={text} onChange={e => setText(e.target.value)} />
      <button onClick={() => setIsPlaying(!isPlaying)}>
        {isPlaying ? 'Pause' : 'Play'}
      </button>
      <VideoPlayer
        isPlaying={isPlaying}
        src="https://interactive-examples.mdn.mozilla.net/media/cc0-videos/flower.mp4"
      />
    </>
  );
}

```

依赖数组改变时 `useEffect` 才会运行。依赖数组可以包含多个依赖项。当指定的所有依赖项在上一次渲染期间的值与当前值完全相同时，React 会跳过重新运行 `useEffect`。

```
useEffect(() => {
  // 这里的代码会在每次渲染后执行
});

useEffect(() => {
  // 这里的代码只会在组件挂载后执行
}, []);

useEffect(() => {
  //这里的代码只会在每次渲染后，并且 a 或 b 的值与上次渲染不一致时执行
}, [a, b]);
```

按需添加清理函数

考虑一个不同的例子。你正在编写一个 ChatRoom 组件，该组件出现时需要连接到聊天服务器。现在为你提供 createConnection() API，该 API 返回一个包含 connect() 与 disconnection() 方法的对象。

在这个例子中就应该指定依赖数组为空数组 []。这告诉 React 仅在组件挂载时运行此代码。

想象 ChatRoom 组件是一个大规模的 App 中许多界面中的一部分。用户切换到含有 ChatRoom 组件的页面上时，该组件被挂载，并调用 connection.connect() 方法连接服务器。然后想象用户此时突然导航到另一个页面，比如切换到“设置”页面。这时，ChatRoom 组件就被卸载了。接下来，用户在“设置”页面忙完后，单击“返回”，回到上一个页面，并再次挂载 ChatRoom。这将建立第二次连接，但是，第一次时创建的连接从未被销毁！当用户在应用程序中不断切换界面再返回时，与服务器的连接会不断堆积。

为了解决这个问题，可以在 Effect 中返回一个 清理（cleanup）函数。

```
useEffect(() => {
  const connection = createConnection();
  connection.connect();
  return () => {
    connection.disconnect();
  };
}, []);
```

每次重新执行 Effect 之前，React 都会调用清理函数；组件被卸载时，也会调用清理函数，清理函数即你 return 的函数。

举例

有时需要添加不是使用 React 编写的 UI 小部件。例如，假设你要向页面添加地图组件，并且它有一个 `setZoomLevel()` 方法，你希望调整缩放级别（zoom level）并与 React 代码中的 `zoomLevel` state 变量保持同步。

```
useEffect(() => {  
  const map = mapRef.current;  
  map.setZoomLevel(zoomLevel);  
}, [zoomLevel]);
```

网络请求获取数据

```
useEffect(() => {  
  let ignore = false;  
  
  async function startFetching() {  
    const json = await fetchTodos(userId);  
    if (!ignore) {  
      setTodos(json);  
    }  
  }  
  
  startFetching();  
  
  return () => {  
    ignore = true;  
  };  
}, [userId]);
```

async 和 await

async：当一个函数被标记为 `async`，这意味着该函数会返回一个 `Promise`。无论函数的返回值是什么，它都会被包装在一个 `Promise` 对象中。如果函数正常执行结束，`Promise` 将被成功地解决（resolved）；如果函数中抛出错误，`Promise` 将被拒绝（rejected）。

await：`await` 关键字用于等待一个 `Promise` 解决其结果。`await` 只能在 `async` 函数内部使用。当执行到 `await` 表达式时，JavaScript 会暂停该 `async` 函数的执行，直到等待的 `Promise` 被解决。一旦 `Promise` 解决，`await` 表达式会返回 `Promise` 的结果，然后函数继续执行。

触发动画

其实也是改变DOM

```
useEffect(() => {  
  const node = ref.current;  
  node.style.opacity = 1; // 触发动画  
  return () => {  
    node.style.opacity = 0; // 重置为初始值  
  };  
}, []);
```

缓存

memo

React 通常在其父组件重新渲染时重新渲染一个组件。你可以使用 `memo` 创建一个组件，当它的父组件重新渲染时，只要它的新 props 与旧 props 相同时，React 就不会重新渲染它。这样的组件被称为**记忆化的** (memoized) 组件。

要记忆化一个组件，就把它包装在 `memo` 中，使用它返回的值替换原来的组件：

```
const Greeting = memo(function Greeting({ name }) {  
  return <h1>Hello, {name}!</h1>;  
});  
  
export default Greeting;
```

React 组件应该始终具有**纯粹的渲染逻辑**。这意味着如果其 props、state 和 context 没有改变，则必须返回相同的输出。通过使用 `memo`，你告诉 React 你的组件符合此要求，因此只要其 props 没有改变，React 就不需要重新渲染。

不过即使使用 `memo`，如果它自己的 state 或正在使用的 context 发生更改，组件也会重新渲染。

以下示例中，`Greeting` 组件在 `name` 更改时重新渲染（因为那是它的 props 之一），但是在 `address` 更改时不会重新渲染（因为它不作为 props 传递给 `Greeting`）

```

'use client';
import { memo, useState } from 'react';

export default function MyApp() {
  const [name, setName] = useState<string>>('');
  const [address, setAddress] = useState<string>>('');
  return (
    <>
      <label>
        Name{' ': ''}
        <input value={name} onChange={e => setName(e.target.value)} />
      </label>
      <label>
        Address{' ': ''}
        <input value={address} onChange={e => setAddress(e.target.value)} />
      </label>
      <Greeting name={name} />
    </>
  );
}

const Greeting = memo(function Greeting({ name }: { name: string }) {
  console.log("Greeting was rendered at", new Date().toLocaleTimeString());
  return <h3>Hello{name && ' ', ''}{name}!</h3>;
});

```

prop 是对象、数组或函数

React 通过浅比较来比较旧的和新的 prop：也就是说，它会考虑每个新的 prop 是否与旧 prop 引用相等。如果每次父组件重新渲染时创建一个新的对象或数组，即使它们每个元素都相同，React 仍会认为它已更改。同样地，如果在渲染父组件时创建一个新的函数，即使该函数具有相同的定义，React 也会认为它已更改。

*基本类型和引用类型（参考指针）

地址	内存单元	变量名
1000	20	x
1004	1	y
1008	155	z
2000	1000	p

知乎 @wztsxyh

useMemo 和 useCallback 可以用来解决这些问题。

useMemo

函数原型

useMemo(calculateValue, dependencies)

```
import { useMemo } from 'react';

function TodoList({ todos, tab, theme }) {
  const visibleTodos = useMemo(() => filterTodos(todos, tab), [todos, tab]);
  // ...
}
```

你需要给 useMemo 传递两样东西：

1. 一个没有任何参数的 calculation 函数，像这样 `() => filterTodos(todos, tab)`，并且返回任何你想要的计算结果。
2. 一个由包含在你的组件中并在 calculation 中使用的所有值组成的依赖列表。

在初次渲染时，你从 useMemo 得到的值将会是你的 calculation 函数执行的结果。

在随后的每一次渲染中，React 将会比较前后两次渲染中的所有依赖项是否相同。如果通过比较所有依赖项都没有发生变化，那么 useMemo 将会返回之前已经计算过的那个值。否则，React 将会重新执行 calculation 函数并且返回一个新的值。

显然 `useMemo` 的一大功能就是**跳过代价昂贵的重新计算**

解决 memo 的问题

```
export default function TodoList({ todos, tab, theme }) {
  // 每次重新渲染时，这将是一个不同的数组.....
  const visibleTodos = filterTodos(todos, tab);
  return (
    <div className={theme}>
      {/* ... 所以List的props永远不会一样，每次都会重新渲染 */}
      <List items={visibleTodos} />
    </div>
  );
}
```

在上面的示例中，`filterTodos` 函数总是创建一个不同数组。这意味着 `List` 的属性永远不会相同，并且你的 `memo` 优化将不起作用。

```
export default function TodoList({ todos, tab, theme }) {
  // 告诉 React 在重新渲染之间缓存你的计算结果...
  const visibleTodos = useMemo(
    () => filterTodos(todos, tab),
    [todos, tab] // ...所以只要这些依赖项不变...
  );
  return (
    <div className={theme}>
      {/* ... List 也就会接受到相同的 props 并且会跳过重新渲染 */}
      <List items={visibleTodos} />
    </div>
  );
}
```

useCallback

函数原型

`useCallback(fn, dependencies)`

基本与 `useMemo` 相同，不过缓存的对象只有函数。`fn` 为想要缓存的函数。
在初次渲染时，`useCallback` 返回你已经传入的 `fn` 函数

在之后的渲染中, 如果依赖没有改变, `useCallback` 返回上一次渲染中缓存的 `fn` 函数; 否则返回这一次渲染传入的 `fn`。

从记忆化回调中更新 state

下面的 `handleAddTodo` 函数将 `todos` 指定为依赖项, 因为它会从中计算下一个 `todos` :

```
function TodoList() {
  const [todos, setTodos] = useState([]);

  const handleAddTodo = useCallback((text) => {
    const newTodo = { id: nextId++, text };
    setTodos([...todos, newTodo]);
  }, [todos]);
  // ...
```

我们期望记忆化函数具有尽可能少的依赖, 当你读取 state 只是为了计算下一个 state 时, 你可以通过传递 updater function 以移除该依赖:

```
function TodoList() {
  const [todos, setTodos] = useState([]);

  const handleAddTodo = useCallback((text) => {
    const newTodo = { id: nextId++, text };
    setTodos(todos => [...todos, newTodo]);
  }, []); // ✅ 不需要 todos 依赖项
  // ...
```

因为这里并不是将 `todos` 作为依赖项并在内部读取它, 而是传递一个关于如何更新 state 的指示器 (`todos => [...todos, newTodo]`) 给 React。

提高：自定义Hook

有兴趣的同学自行了解

[自定义Hook](#)

fetch

Fetch API 可以简洁地发起 HTTP 请求, 并处理服务器的响应。

Fetch API 基于 Promise 设计，使得异步操作更加简洁和易于理解。

基本用法

```
fetch(url)
  .then(response => response.json()) // 解析 JSON 数据
  .then(data => console.log(data))   // 处理数据
  .catch(error => console.error('Error:', error)); // 错误处理
```

返回一个 Promise 对象，Promise 在请求成功时返回 Response 对象，在请求失败时被拒绝。

参数

url：要发送请求的目标 URL。

options（可选）：可以指定请求方法（GET、POST 等）、请求头、请求体等内容。

```
fetch('https://api.example.com/data', {
  method: 'POST', // 指定请求方法
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    key: 'value'
  })
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error('Error:', error));
```

fetch 是**异步**的

```

'use client';
import { useState, useEffect, ChangeEventHandler } from "react";

export default function App() {
  type student = { data: { xm: string, zymc: string, sj: string, email: string } }
  const [info, setInfo] = useState<student>({ data: { xm: '', zymc: '', sj: '', email: '' } });
  const [inputValue, setInputValue] = useState('');

  useEffect(() => {
    if (parseInt(inputValue) >= 1000000000 && parseInt(inputValue) < 9999999999) {
      fetch('http://kyjs.zju.edu.cn/srts/prod-api/user/xsDetailsByxh?xh=' + inputValue)
        .then(response => response.json())
        .then((data) => {
          if (typeof data.data.xm !== undefined) {
            setInfo(data);
          }
        })
        .catch(error => console.error('Error fetching data:', error));
    }
  }, [inputValue]);

  const handleInputChange: ChangeEventHandler = (event: React.ChangeEvent<HTMLInputElement>) =>
    setInputValue(event.target.value);
};

return (
  <div>
    <div>姓名: {info.data.xm}</div>
    <div>专业: {info.data.zymc}</div>
    <div>手机号: {info.data.sj}</div>
    <div>邮箱: {info.data.email}</div>
    <input
      type="text"
      value={inputValue}
      onChange={handleInputChange}
    />
  </div>
);
}

```